

CNN Image Classification Project

CIFAR-10 — 10-category image classifier using a Convolutional Neural Network

1. Project Overview

This project builds an image classification model using a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset. CIFAR-10 contains 60,000 real-world color photos evenly split across 10 everyday categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The goal is to take a raw photo as input and correctly predict which of the 10 categories it belongs to.

Unlike the earlier ANN project (NSL-KDD network intrusion detection), which worked on structured tabular data, this project works on unstructured image data, requiring convolution, ReLU, and pooling layers to extract spatial visual features before handing off to a standard ANN (dense layers) for final classification.

2. Dataset Selection

What: CIFAR-10, a balanced 10-class image dataset (5,000 training images per class, 6,000 total per class including test), built directly into Keras.

Why: CIFAR-10 offers real, recognizable photos (not abstract data), is small enough (32x32 pixels) to train a from-scratch CNN within a single session, and is a well-established benchmark for demonstrating CNN fundamentals in a portfolio.

3. Loading the Dataset

```
from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

Output:

```
(50000, 32, 32, 3)
(50000, 1)
(10000, 32, 32, 3)
(10000, 1)
```

`cifar10.load_data()` returns two pairs: training data (50,000 images used to teach the model) and test data (10,000 images held back, never shown during training, used only to grade the model at the end). Each image is 32x32 pixels with 3 color channels (red, green, blue). Each label is a single number from 0-9 representing the image's category.

4. Exploratory Data Analysis (EDA)

```
import matplotlib.pyplot as plt

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
plt.show()
```

Checking class balance:

```
import numpy as np

unique, counts = np.unique(y_train, return_counts=True)
print(dict(zip(unique, counts)))
```

Result: every one of the 10 categories has exactly 5,000 training images — the dataset is perfectly balanced, so the model has no reason to be biased toward any single category.

5. Preprocessing

What: Scale pixel values from the 0-255 range down to 0-1, and convert labels from single numbers into one-hot encoded vectors.

Why: Neural networks train faster and more reliably on small, consistent input ranges (same reasoning as feature scaling in the ANN project). One-hot encoding is required because the model's output layer produces 10 separate probabilities, so the true label must be shaped the same way for comparison during training.

```
x_train = x_train / 255.0
x_test = x_test / 255.0

from tensorflow.keras.utils import to_categorical

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

Example of one-hot encoding: label 8 (ship) becomes [0,0,0,0,0,0,0,0,1,0] — a list of 10 zeros with a single 1 placed at position 8, matching the label's own number.

6. Building the CNN Architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

```

Architecture logic: two rounds of convolution + ReLU + pooling progressively extract visual patterns (round 1 finds simple patterns like edges; round 2 combines those into more complex shapes), followed by Flatten (unrolls the final feature maps into one list) and two Dense layers (the ANN 'tail' that makes the final decision), ending in a 10-unit softmax layer that outputs a probability per category.

model.summary() output:

Layer	Output Shape	Params
Conv2D (32 filters, 3x3)	(None, 30, 30, 32)	896
MaxPooling2D (2x2)	(None, 15, 15, 32)	0
Conv2D (64 filters, 3x3)	(None, 13, 13, 64)	18,496
MaxPooling2D (2x2)	(None, 6, 6, 64)	0
Flatten	(None, 2304)	0
Dense (64, relu)	(None, 64)	147,520
Dense (10, softmax)	(None, 10)	650

Total trainable parameters: 167,562

7. Compiling and Training

```

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=10, batch_size=64,
                    validation_data=(x_test, y_test))

```

adam handles the backpropagation weight updates; categorical_crossentropy measures error since labels are one-hot across 10 classes; batch_size=64 means weights update after every 64 images rather than after each single image or the full 50,000 at once.

Epoch	Train Accuracy	Val Accuracy	Val Loss
1	45.8%	56.6%	1.224
3	64.2%	65.3%	1.003
5	69.6%	67.4%	0.946
8	74.5%	70.5%	0.877
10	76.7%	69.6%	0.909

Note: validation accuracy plateaued and slightly dipped in the final two epochs while training accuracy kept climbing — an early sign of mild overfitting, discussed further under Model Limitations.

8. Model Evaluation

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print('Test accuracy:', test_accuracy)
```

Result: Test accuracy = 69.6%

```
import numpy as np
from sklearn.metrics import classification_report

y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

print(classification_report(y_true, y_pred_classes, target_names=class_names))
```

Category	Precision	Recall	F1-score
airplane	0.70	0.77	0.74
automobile	0.89	0.72	0.80
bird	0.72	0.47	0.57
cat	0.58	0.43	0.49
deer	0.68	0.62	0.65
dog	0.53	0.74	0.62
frog	0.73	0.80	0.76
horse	0.82	0.69	0.75

ship	0.74	0.86	0.80
truck	0.67	0.86	0.75

9. Deployment with Streamlit

What: A simple web app where a user uploads any photo and receives the model's predicted category and confidence score.

```
import streamlit as st
import numpy as np
from tensorflow.keras.models import load_model
from PIL import Image

model = load_model('cifar10_cnn_model.h5')
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

st.title('CNN Image Classifier - CIFAR-10')
uploaded_file = st.file_uploader('Choose an image', type=['jpg', 'jpeg', 'png'])

if uploaded_file is not None:
    image = Image.open(uploaded_file).convert('RGB')
    st.image(image, caption='Uploaded photo', width='stretch')

    image_resized = image.resize((32, 32))
    image_array = np.array(image_resized) / 255.0
    image_array = np.expand_dims(image_array, axis=0)

    prediction = model.predict(image_array)
    predicted_class = class_names[np.argmax(prediction)]
    confidence = np.max(prediction) * 100

    st.write(f'Prediction: {predicted_class}')
    st.write(f'Confidence: {confidence:.2f}%')
```

Real-world test results (photos outside the CIFAR-10 dataset itself):

Test Image	Prediction	Confidence	Correct?
Stylized illustration (person on bicycle)	frog	57.2%	No — not a real CIFAR-10 category
Real airplane photo (close-up, clouds)	ship	63.8%	No
Real cruise ship photo	automobile	95.4%	No — confidently wrong
Real bird photo (hummingbird, wings spread)	airplane	45.3%	No — low confidence
Real horse photo (running, dark background)	dog	31.6%	No — low confidence

10. Model Limitations

- Overall test accuracy is moderate (~70%), reflecting a relatively shallow architecture (only two convolution+pooling blocks) compared to modern deep architectures like ResNet or VGG.
- Weakest categories are cat (F1 0.49) and bird (recall 0.47) — likely due to visual similarity with other animal classes (dog, deer) once images are shrunk to 32x32 and fine texture detail is lost.
- The model confuses categories that share color/background patterns rather than true object understanding — e.g., airplane vs. ship, both often photographed against large blue backgrounds.
- The model can be confidently wrong: a cruise ship photo was classified as 'automobile' with 95% confidence, showing that high confidence does not guarantee correctness.
- Performance degrades on real-world photos that differ from CIFAR-10's training distribution — unusual camera angles, close-up crops, action shots, or photos containing categories not in the original 10 (e.g., people, bicycles, illustrations).
- Mild overfitting was observed: validation accuracy plateaued/dipped slightly in the final training epochs while training accuracy kept increasing, suggesting the model was starting to memorize training-set specifics.
- Every input image is downsampled to 32x32 pixels before prediction, which discards significant detail from typical modern smartphone photos, further limiting real-world accuracy.

11. Key Terms Glossary

Term	Meaning
Convolution	Sliding a small grid of numbers (filter) across an image, multiplying and summing overlapping values, to produce a score showing how well each patch matches a pattern.
Filter / Kernel	The small grid of numbers (e.g. 3x3) used in convolution. Starts random, gets tuned during training via backpropagation.
Feature map	The output grid produced by sliding one filter across the entire image — shows where a specific pattern was detected.
ReLU	Activation function applied after convolution: keeps positive values unchanged, converts all negative values to 0.
Pooling (Max pooling)	Shrinks a feature map by keeping only the largest value in each small non-overlapping patch, reducing size while keeping the strongest signals.
Flatten	Reshapes multiple small feature-map grids into one single long list of numbers, with no math performed — just rearranging shape.
Dense layer	A standard fully-connected ANN layer, identical to the layers used in the NSL-KDD ANN project.
Softmax	Final-layer activation that converts raw output numbers into probabilities across all classes, summing to 100%.
One-hot encoding	Converts a single category number into a list of 0s with a single 1 at the position matching that category.
Epoch	One complete pass of the entire training dataset through the model.

Batch size	Number of training images processed together before the model's weights are updated once.
Overfitting	When a model performs increasingly well on training data but stops improving (or worsens) on unseen validation/test data.

12. Interview Answer

“I built an image classification CNN using the CIFAR-10 dataset, which contains 60,000 real-world photos across 10 everyday categories such as airplanes, cars, and animals. I designed a convolutional neural network with two convolution-ReLU-pooling blocks to progressively extract visual features from raw pixels — the first block detecting simple patterns like edges, the second combining those into more complex shapes — followed by a flatten layer and dense layers ending in a softmax output for 10-class classification. After training for 10 epochs, the model reached roughly 70% test accuracy, performing strongest on vehicle categories like automobile, ship, and truck, and weaker on visually similar animal categories like cats and birds. I deployed the trained model in a Streamlit web app that lets a user upload any photo and see the model's prediction and confidence score in real time, and I deliberately stress-tested it with real-world photos outside the training distribution — including out-of-category images, unusual angles, and visually ambiguous sub-types — to understand and document its practical limitations, such as confidently-wrong predictions and sensitivity to background and framing.”